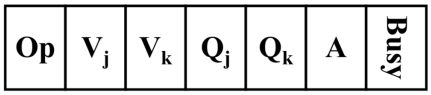
1 Lesson 20-10-26

In the last lesson we saw the structure of the Tomasulo architecture. Now we look into the reservation stations and the register files in more details.

1.0.1 Fields

First off, the reservation station's records are made out of the following *fields*:



These are:

- O_p: represents the operation being performed on operands;
- Q_j, Q_k: the reservation station that will produce the relevant source operand (0 indicates the value is in V_j, V_k);
- V_j, V_k: the value of the source operands;
- A: used to hold the memory address information for a load or store;
- Busy: 1 if occupied, 0 if not occupied.

Information about which register is where is contained within the register file (floating point or integer): we store a Q_i flag, which records the reservation station whose result should be stored in this register (if blank or 0, no values are destined for this register).

1.0.2 Loads and stores

Load and store instructions can be executed in any order, provided that they do not refer to the same address. In this case, by exchanging their order:

- If the load is before the store, a WAR hazard may happen;
- If the store is before the load, a RAW hazard may happen.

To avoid hazards coming from reordering load and store instructions, the processor must:

- Compute memory addresses in order;
- Perform some check for every load or store instruction already in the buffers.

More formally:

- When a **load** is about to be issued, the store buffer is checked for store instructions acting on the same address, and the load is not issued until these previous instructions complete;
- When a **store** is about to be issued, both the load and store buffers are checked for load and store instructions acting on the same address, and the load is not issued until these previous instructions complete;

1.1 Speculation

This section follows pretty naturally from what we've seen so far:

- First off we introduced the bare, unpipelined, processor;
- Then we introduced the *pipeline*, and saw that hazards appear;
- When more than one execution unit was needed, we issued to pipelined stages via an issue unit in the *Tomasulo architecture*. This showed that hazards were really dependencies showing up has hardware constraints.

We know extend the Tomasulo architecture by introducing **speculation**.

So far, when we met a branch instruction in the Tomasulo architecture, we stopped issuing instruction until the branch was taken. We saw that when pipelining some kind of speculation could have been done on the branch outcome. Let's see if we can do the same here.

Hardware-based speculation is a technique for reducing the effects of control dependences in a processor implementing dynamic scheduling. If a processor supports branch prediction with dynamic scheduling, it fetches and issues instructions, as if the branch prediction was always correct. If a processor also supports hardware-based speculation, it also *executes* them.

It combines three ideas:

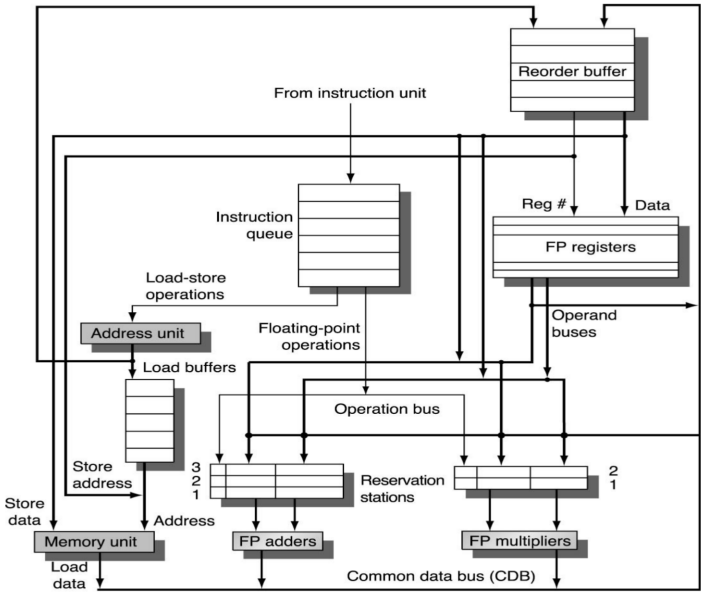
- **Dynamic branch prediction**: we assume the result of branch instructions before they finish executing;
- **Dynamic scheduling**: we schedule instructions when they are *ready* to execute;
- **Speculation**: we start executing the predicted instructions before the branch finishes executing..

In such a way, the processor implements **data flow execution**: operations execute as soon as their operands are available.

The basic Tomasulo's architecture is adopted, extending it to support speculation. There are two different steps in instruction execution:

- The computation of results and their bypassing to other instructions;
- The update of register file and memory, which is only performed when the instruction is no longer speculative (*instruction commit* via the **ROB** (*ReOrder Buffer*)); in this way, in-order commitment is implemented.

The ROB provides additional virtual registers and integrates the store buffer existing in the original Tomasulo's architecture.



1.1.1 Fields

Each ROB record has 4 *fields*:

- Instruction type: branch, store, or register;
- Destination: register number, or memory address;
- Value: contains the value when the instruction has completed but still didn't commit to a register file;
- Ready: indicates whether the instruction completed its execution.

With speculation, data for an incoming instruction may be read:

- From the ROB, if the producing instruction didn't commit yet (because a branch wasn't confirmed);
- From the register file, otherwise (this is the normal execution path we saw this far).

1.1.2 How does it work?

An instruction is extracted from the instruction queue (that is, **issued**) if there is:

- An empty reservation station;
- An empty slot in the reorder buffer.

If this is not the case, the instruction issue is stalled. Otherwise:

- The operands for the instruction are sent to the reservation station, if they are in the register file or in the reorder buffer.
- The number of the reorder buffer entry for the instruction is sent to the reservation station to tag the instruction (and its results, when they will be written on the **CDB** (*Common Data Bus*)).

At this point we want to know when this instruction will be **executed**. The instruction is executed as soon as all the required operands are available: this avoids any RAW hazard. Operands are possibly taken from the CDB as soon as another instruction produces them. The length of this step of course varies depending on the instruction type (and thus the execution unit that gets the instruction).

Next it's time for **writeback**. As soon as it is available, the result is put on the CDB (together with the tag identifying the instruction) and sent to the Reorder Buffer. Any reservation station waiting for the result reads it, and the reservation station entry is marked as available

Why does this ensure control hazards are avoided? The Reorder Buffer is ordered according to instructions original order. As soon as one instruction reaches the head of the buffer;

- If it is a mispredicted branch, the buffer is flushed, and the execution is restarted with the correct successor of the instruction;
- Otherwise, the result is written in the register file or in memory (in case of a store);
- In both cases, the Reorder Buffer entry is marked as free. The Reorder Buffer is implemented as a circular buffer.

The result of this is:

- **WAR** and **WAW** name dependencies don't result in hazards, since dynamic renaming is implemented and memory updating occurs in-order (i.e., when a store reaches the top of the ROB).

  Register renaming is implemented via the **RAT** (*Register Alias Table*) which assigns a physical register to a logical one. The RAT is updated every time an instruction is committed to a register.

- **RAW** hazards in memory are prevented by:
  - Enforcing the program order while computing the effective address of a load / store;
  - Preventing a load to initiate its second step if any active ROB entry occupied by a store has a Destination field matching the A field of the load.

  RAW hazards in registers are of course prevented by the Tomasulo architecture itself.